

03分治

代码题

Q1 男女生舞伴

假设 n 个学生 $S[i]$ 的身高 $height[i]$ 不同 ($1 \leq i \leq n$), 并且已排序为 $height[1] < height[2] < \dots < height[n]$ 。另外, 他们的性别对应地记录在数组 $sex[1..n]$ 中, $sex[i] = F$ 表示 $S[i]$ 是女生, $sex[i] = M$ 表示 $S[i]$ 是男生 ($1 \leq i \leq n$)。如果 $sex[i] = F$, $sex[j] = M$, 并且 $height[i] < height[j]$, 那么 $S[i]$ 和 $S[j]$ 可组成一对合格的舞伴。请用分治法设计一个复杂度为 $O(n)$ 的算法来计算在这 n 个学生中有多少个不同的合格的配对方案?

ppl-ai-file-upload.s3.amazonaws

基本思想

这道题的条件可以简化为统计满足以下两个条件的点对 (i, j) 的数量：

1. 性别条件: $sex[i] = F$ 且 $sex[j] = M$ 。
2. 身高条件: $height[i] < height[j]$ 。

由于题目已知数组是按身高升序排列的 ($i < j \implies height[i] < height[j]$)，所以“身高条件”等价于索引关系 $i < j$ 。

因此，问题等价于：统计数组中满足 $i < j$ 且 $sex[i] = F, sex[j] = M$ 的对数。

这是一个典型的顺序对统计问题（类似于逆序对，但这题更简单，甚至可以一遍扫描，但题目强制要求用分治法）。

分治策略：

1. 分解 (Divide)：将学生数组从中间切开，分为左半部分 L 和右半部分 R 。
2. 解决 (Conquer)：递归计算 L 内部的配对数，以及 R 内部的配对数。
3. 合并 (Combine)：计算跨越左右两部分的配对数。
 - 跨越的配对必须是：左边选一个女生 ($i \in L$)，右边选一个男生 ($j \in R$)。
 - 因为整个数组按身高排序，所以 $\forall i \in L, \forall j \in R$ ，都有 $height[i] < height[j]$ 成立。
 - 所以跨越的配对数 = (左半部分的女生数量) $\times$ (右半部分的男生数量)。

复杂度：

$T(n) = 2T(n/2) + O(1)$ ，根据 Master 定理， $T(n) = O(n)$ 。

(注：如果每次合并都重新扫描统计 F/M 数量，合并代价是 $O(n)$ ，总复杂度是 $O(n \log n)$ 。为了达到 $O(n)$ ，需要在递归返回时直接返回该区间内的 F 和 M 的统计信息，使合并代价降为 $O(1)$ 。)

代码块

```
1  算法 CountPairs(sex, left, right)
2  输入：性别数组 sex，当前区间的左右边界 left, right
3  输出：一个元组 (pairs, countF, countM)
4      pairs: 该区间内的合格配对总数
5      countF: 该区间内的女生(F)总数
6      countM: 该区间内的男生(M)总数
7
8  1. If left == right:
9      If sex[left] == 'F':
10         Return (0, 1, 0) // 0对, 1女, 0男
11     Else:
12         Return (0, 0, 1) // 0对, 0女, 1男
13
14  2. mid = (left + right) / 2
15
```

```

16 3. // 递归求解左右两半
17     (pairsL, fL, mL) = CountPairs(sex, left, mid)
18     (pairsR, fR, mR) = CountPairs(sex, mid + 1, right)
19
20 4. // 合并步骤
21     // 跨越配对数 = 左边的女生数 * 右边的男生数
22     crossPairs = fL * mR
23
24     // 总配对数 = 左边配对 + 右边配对 + 跨越配对
25     totalPairs = pairsL + pairsR + crossPairs
26
27     // 当前区间的总女生数和男生数
28     totalF = fL + fR
29     totalM = mL + mR
30
31 5. Return (totalPairs, totalF, totalM)

```

代码块

```

1  #include <iostream>
2  #include <vector>
3
4  using namespace std;
5
6  // 定义一个结构体来返回多个值
7  struct Result {
8      long long pairs; // 配对总数
9      int countF;      // 区间内女生F的数量
10     int countM;      // 区间内男生M的数量
11 };
12
13 Result countPairs(const vector<char>& sex, int left, int right) {
14     // Base Case: 只有一个元素
15     if (left == right) {
16         if (sex[left] == 'F') {
17             return {0, 1, 0}; // 0对, 1个F, 0个M
18         } else {
19             return {0, 0, 1}; // 0对, 0个F, 1个M
20         }
21     }
22
23     int mid = (left + right) / 2;
24
25     // 递归计算左右两半
26     Result L = countPairs(sex, left, mid);
27     Result R = countPairs(sex, mid + 1, right);

```

```

28
29     Result current;
30     // 1. 统计当前区间的总F和总M (用于上一层递归使用)
31     current.countF = L.countF + R.countF;
32     current.countM = L.countM + R.countM;
33
34     // 2. 计算跨越配对数
35     // 题目已知 height 已排序, 所以左边的任意 F 身高一定 < 右边的任意 M
36     // 跨越配对 = (左边F的数量) * (右边M的数量)
37     long long crossPairs = (long long)L.countF * R.countM;
38
39     // 3. 总配对数 = 左内部 + 右内部 + 跨越
40     current.pairs = L.pairs + R.pairs + crossPairs;
41
42     return current;
43 }
44
45 int main() {
46     // 题目条件: height 必须是升序的
47     // height = {150, 155, 160, 165, 170}; (隐含条件, 代码中甚至不需要用到 height
48     // 数组)
49     // 对应 sex = {'F', 'F', 'M', 'F', 'M'};
50     // 注意: 你原来的 main 函数里的 height {150, 160, 155...} 不是升序的, 违反了题目
51     // 假设。
52     // 如果题目说 height 没排序, 那复杂度就得是  $O(n \log n)$  (类似逆序对)。
53     // 这里我们严格遵循题目“已排序”的假设。
54
55     vector<char> sex = {'F', 'F', 'M', 'F', 'M'};
56     // F(1), F(2), M(3), F(4), M(5)
57     // F1 配 M3, M5
58     // F2 配 M3, M5
59     // F4 配 M5
60     // 总共 5 对
61
62     int n = sex.size();
63     Result res = countPairs(sex, 0, n - 1);
64
65     cout << "Total Pairs: " << res.pairs << endl; // 应输出 5
66
67     return 0;
68 }

```